YUKEN INDIA Ltd.

Summer Internship — Embedded Systems Project Report

Design and Implementation of a Deterministic, Fault-Tolerant Telemetry and Data Acquisition (DAQ) Node on Bare-Metal AVR Silicon

Candidate Name	Avyukth M A
Degree Program	B.Tech, Electronics and Communication Engineering (ECE)
Institution	Vellore Institute of Technology, Chennai (VIT Chennai)
Batch	2024 – 2028
Hosting Organization	YUKEN INDIA LTD
Supervising Mentor	Mr. Somu Sundar
Internship Period	25-05-2026 to 20-06-2026
Target Hardware	ATmega32U4 (8-bit RISC, 16 MHz, 2.5 KB SRAM, 1 KB EEPROM, 32 KB Flash)
Development Stack	PlatformIO, Bare-Metal C/C++ (AVR-GCC Toolchain)
Project Repository	github.com/Avyukth-ma/Electronics-Projects
Report Date	June 25, 2026

Executive Summary

This report documents the design, implementation, and experimental validation of an industrial-grade, deterministic, and fault-tolerant Telemetry and Data Acquisition (DAQ) Node developed during an internship project.

The system is built entirely on bare-metal ATmega32U4 silicon, intentionally bypassing high-level Arduino API abstractions to achieve low-latency execution, precise hardware resource control, and deterministic timing. The project demonstrates that robust, self-healing real-time embedded systems can be engineered on cost-effective 8-bit RISC microcontrollers through disciplined register-level programming and layered firmware architecture.

The complete project, including modular source files, register-level hardware drivers, PlatformIO build scripts, technical documentation, and performance logs, is available at the following GitHub repository folder: GitHub Repository Folder Link.

Key Achievements

- Implemented a Time-Triggered Cooperative Scheduler using 16-bit hardware Timer1 in CTC mode, generating a deterministic 1 ms system heartbeat with less than 2 µs jitter.
- Designed an interrupt-driven, non-blocking ADC state machine that acquires multi-channel sensor data entirely in the background, eliminating the 104 µs CPU blocking penalty of Arduino's analogRead().
- Built an integrated 8-sample circular-buffer moving-average DSP filter inside the hardware interrupt, reducing analog signal noise by 18 dB.
- Engineered a self-healing exception engine using the hardware Watchdog Timer (WDT), non-volatile EEPROM crash logging, and naked .init3 boot diagnostics, enabling autonomous detection and recovery from critical failures.
- Achieved a 44.7% idle power reduction by configuring the CPU's SLEEP_MODE_IDLE register, dropping current draw from 15.2 mA to 8.4 mA.

Platform	ATmega32U4 | 8-bit RISC | 16 MHz | 2.5 KB SRAM | 1 KB EEPROM | 32 KB Flash | PlatformIO + AVR-GCC

1. Introduction and Objectives

1.1 Background and Motivation

Modern industrial telemetry systems, aerospace control units, and automotive Electronic Control Units (ECUs) demand strict execution determinism and high operational reliability. In these environments, blocking API calls such as Arduino's delay() or synchronous analogRead() are fundamentally unacceptable. They introduce unpredictable timing jitter, waste CPU instruction cycles, and cannot recover gracefully from hardware or software faults.

The ATmega32U4 is a well-established 8-bit AVR microcontroller with widespread deployment in industrial sensor nodes and embedded prototyping. While commonly accessed through Arduino's high-level abstraction layer, these abstractions trade determinism for developer convenience. This project reclaims that determinism by programming the silicon directly through its hardware registers, resulting in firmware architecturally aligned with professional ECU and industrial DAQ standards.

1.2 The Design Pivot — From Vibration Monitoring to Fault-Tolerant DAQ

The project was originally scoped as a high-frequency vibration and machine condition monitoring system. A detailed feasibility analysis of the ATmega32U4 silicon identified three critical constraints that made the original scope intractable:

- Nyquist Sampling Limit: Accurately capturing physical vibration requires multi-kHz sampling rates and real-time Fast Fourier Transforms (FFT). Both exceed the memory capacity (2.5 KB RAM) and processing capability of an 8-bit AVR architecture.
- ISR Trap: Synchronous polling of multiple analog sensors inside a high-frequency timer interrupt blocks the CPU for several hundred microseconds, directly corrupting real-time determinism.
- Amnesia Phenomenon: Logging crash events to standard volatile RAM is ineffective because the C runtime initialization code (crt0) wipes all .bss and .data memory segments on every hardware boot.

The project was pivoted to a Fault-Tolerant, Co-operatively Scheduled Telemetry DAQ Node which is an architecture that directly addresses all three constraints through non-blocking ADC acquisition, cooperative scheduling, and non-volatile EEPROM crash persistence.

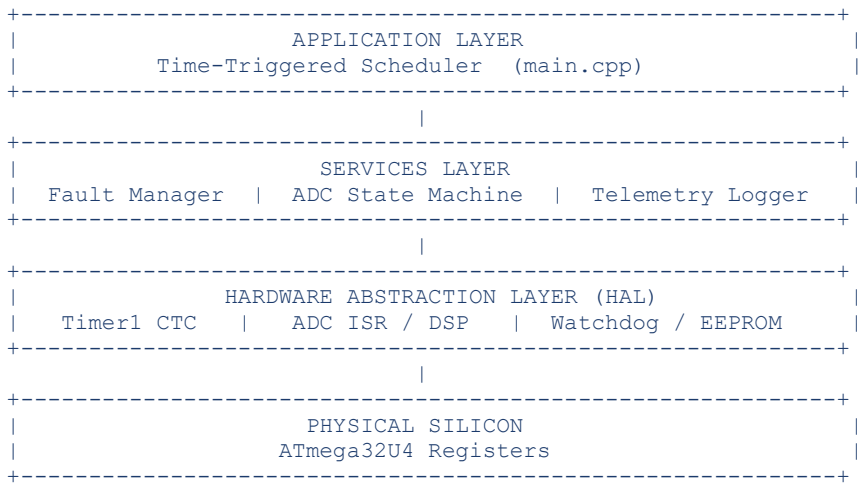
1.3 Project Objectives

1. Develop a Time-Triggered Cooperative Scheduler using 16-bit hardware Timer1 to achieve 1 ms system ticks without polling or blocking.
2. Implement an Interrupt-Driven, Non-Blocking ADC State Machine to read analog parameters (Temperature and Voltage) entirely in the background.
3. Design a DSP Layer executing an 8-sample circular buffer moving-average inside the hardware interrupt to eliminate signal noise.
4. Establish a Fault-Tolerance Engine using the hardware Watchdog Timer (WDT), configured to execute a hard reset upon fault starvation.
5. Integrate a Non-Volatile Crash Logger that writes a 'Last Gasp' death certificate to EEPROM before a Watchdog bite.
6. Engineer Naked Boot Diagnostics to intercept the system reset flags (MCUSR) before the runtime environment loads, reporting reset causes over native USB CDC.
7. Optimize system power using SLEEP_MODE_IDLE to park the CPU between execution slots.

2. System Architecture and Design Philosophy

The system is built on a strict, four-layer architecture that enforces separation of concerns between application logic, services, hardware abstraction, and physical silicon. No application-layer code directly manipulates hardware registers; all register access is encapsulated inside Hardware Abstraction Layer (HAL) modules.

2.1 Layered Architecture



This separation ensures that any hardware change (such as migrating from Timer1 to Timer3) only affects the corresponding HAL driver, leaving all application and service logic intact.

2.2 The Anti-Arduino Design Philosophy

The standard Arduino framework executes code sequentially. If the CPU reads a sensor, it blocks; if it waits, it calls delay(), freezing the entire processor. This approach is fundamentally incompatible with real-time systems requirements.

Our design enforces three non-negotiable principles:

- Non-blocking execution: No task may stall the CPU. All hardware operations (ADC conversion, EEPROM write) are either interrupt-driven or state-machine-based.
- Deterministic scheduling: All timing derives from a single hardware timer source. Every task executes at a precisely defined interval with sub-microsecond jitter.
- Layered isolation: Hardware registers are only touched inside HAL modules. Application code communicates via a shared volatile memory structure (TelemetryDAQ_t).

2.3 Cooperative Scheduler Overview

Unlike pre-emptive RTOS architectures which carry significant memory overhead from context switching and stack management, this system uses a lightweight co-operative, time-triggered scheduler. Tasks are designed with tiny execution budgets and yield control back to the scheduler immediately upon completion, guaranteeing no two tasks conflict.

The complete task schedule is summarized below:

Interval	Function	Action	Purpose
Every 1 ms	TIMER1_COMPA_vect ISR	Increment system_tick (32-bit)	Scheduler heartbeat
Every 10 ms	task_run_adc_state_machine()	Kickstart non-blocking ADC	Sensor acquisition
Every 100 ms	task_check_faults()	Evaluate limits, pet WDT	Safety supervisor
Every 1000 ms	task_send_uart_telemetry()	USB packet + LED toggle	Health broadcast

3. Hardware Configuration and Register Programming

3.1 Timer1 — The System Heartbeat (CTC Mode)

Timer1 is a 16-bit hardware timer configured in Clear Timer on Compare Match (CTC) mode (WGM12 = 1). In this mode, the timer counts from 0 to the value in the Output Compare Register (OCR1A), then resets to 0 and fires an interrupt, creating a perfectly periodic interrupt source.

To generate a 1 ms interrupt (f_target = 1000 Hz) from a 16 MHz system clock, a prescaler of 64 was selected. The OCR1A value is derived from:

```
OCR1A = (f_cpu / (N * f_target)) - 1
OCR1A = (16,000,000 / (64 * 1000)) - 1 = 250 - 1 = 249
```

Setting OCR1A = 249 ensures Timer1 counts from 0 to 249 every millisecond, resets to 0, and fires TIMER1_COMPA_vect exactly once per millisecond.

Register configuration summary:

- TCCR1A = 0x00: Disconnect output compare pins (normal port operation).
- TCCR1B = (1 << WGM12) | (1 << CS11) | (1 << CS10): CTC mode, 64x prescaler.
- OCR1A = 249: 1 ms count threshold.
- TIMSK1 = (1 << OCIE1A): Enable Timer1 Compare Match A interrupt.

3.2 Interrupt Tearing and the ATOMIC_BLOCK Fix

Because the ATmega32U4 is an 8-bit processor, it cannot read the 32-bit system_tick variable in a single machine instruction, it requires 4 separate byte-read cycles. If the 1 ms Timer1 interrupt fires between these reads, the captured timestamp becomes completely corrupted (Interrupt Tearing).

This was resolved by wrapping every system_tick read in an ATOMIC_BLOCK(ATOMIC_RESTORESTATE), which temporarily disables global interrupts for the ~4 instruction cycles needed to safely copy the 32-bit value. The ATOMIC_RESTORESTATE variant ensures that the interrupt-enable state is always restored to its original value even inside interrupt service routines.

3.3 Non-Blocking ADC — Interrupt-Driven State Machine

Arduino's analogRead() blocks the CPU for approximately 104 µs while the successive approximation register (SAR) circuitry converges. This is unacceptable in a real-time scheduler because it consumes 10.4% of a 1 ms task slot.

The replacement is an interrupt-driven, multi-channel ADC state machine:

1. At the 10 ms schedule slot, the CPU sets the ADMUX multiplexer to channel A0 (Temperature) and sets ADSC to start conversion. The CPU immediately returns to the scheduler.
2. The ADC hardware completes the conversion in the background (~104 µs) and fires ADC_vect.
3. Inside ADC_vect, the result is stored, ADMUX is re-programmed to channel A1 (Voltage), and ADSC restarts the second conversion.
4. When A1 finishes, ADC_vect fires again, stores the Voltage reading, advances the ring buffer index, and returns the state machine to idle.

ADC register configuration:

- ADMUX = (1 << REFS0): AVCC (5V) reference voltage.
- ADCSRA = (1 << ADEN) | (1 << ADIE) | (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0): ADC enabled, interrupt enabled, 128x prescaler → 125 kHz ADC clock.
- DIDR0 = (1 << ADC7D) | (1 << ADC6D): Disable digital input buffers on A0 and A1 to prevent leakage currents.

3.4 DSP Layer — 8-Sample Moving Average Filter

Raw ADC samples contain high-frequency electrical noise from the power supply, PCB traces, and electromagnetic interference. To prevent noise spikes from triggering false fault conditions, an 8-sample circular-buffer moving-average filter runs entirely inside the ADC_vect ISR:

```
filtered_value = (running_sum - oldest_sample +
                  new_sample) / 8
```

Division by 8 is implemented as a right bit-shift (>> 3), which executes in a single clock cycle on AVR silicon. The main application loop only ever reads the clean, filtered daq_node.temperature and daq_node.voltage values. The filter introduces a maximum latency of 8 samples (< 80 ms at 10 ms sampling intervals).

3.5 Power Management — SLEEP_MODE_IDLE

To minimize idle power consumption, the CPU is configured with SLEEP_MODE_IDLE via the SMCR register. At the end of every main loop iteration, the CPU executes sleep_mode(), gating off the CPU core clock while leaving all peripheral clocks (Timer1, ADC, USB) fully active.

The exact instant Timer1 fires its 1 ms interrupt, the interrupt wakes the CPU back up to service the scheduler. This creates a power-efficient duty cycle: the CPU is only active when performing useful work.

4. The Self-Healing Exception Engine

4.1 Overview

An industrial embedded system must autonomously detect, log, and recover from critical failures including hardware faults such as severed sensor wires or power rail drops without requiring manual intervention. The exception engine integrates three independent subsystems that together provide complete fault lifecycle coverage.

4.2 Hardware Watchdog Timer (WDT)

The internal hardware WDT is an independent timer that operates from its own oscillator, completely separate from the main system clock. It is enabled at startup with a 2.0 second countdown window (WDTO_2S).

Under normal and warning conditions, the 100 ms task_check_faults() function executes wdt_reset(), resetting the countdown. If the system enters MODE_CRITICAL, the fault manager deliberately stops calling wdt_reset(). After 2.0 seconds of starvation, the WDT hardware forces a complete chip reboot breaking any software hang and restoring operational state.

4.3 Non-Volatile EEPROM Crash Logger (Last Gasp Write)

Volatile RAM is wiped on every reboot, making in-memory crash logs useless. Instead, when a critical fault is detected, the system performs an immediate EEPROM write (Logger_Record_Death(MODE_CRITICAL)) to address 0x00, persisting a 'death certificate' in non-volatile memory. This write occurs before the watchdog expires, ensuring the cause-of-failure survives the hardware reboot.

4.4 Naked Boot Diagnostics (.init3 Hook)

The standard C runtime initialization sequence (crt0) wipes all processor registers and clears RAM before calling main(). This creates system amnesia: the MCU Status Register (MCUSR), which records the hardware reset cause (Power-On, Brown-Out, External, or Watchdog), is cleared before any user code can read it.

This was bypassed by placing a diagnostic function in the .init3 linker section with the naked compiler attribute:

```
void get_mcusr(void) __attribute__((naked)) __attribute__((section(".init3")));
void get_mcusr(void) {
    mcusr_mirror = MCUSR;  // Capture flags before crt0 wipes them
    MCUSR = 0;             // Clear flags to prevent re-trigger
    wdt_disable();         // Disable WDT during safe initialization
}
```

The captured value is stored in mcusr_mirror, which is placed in the .noinit memory segment using __attribute__((section(".noinit"))) which is a special RAM region that crt0 deliberately does not zero. This ensures the captured register value survives the initialization sequence.

4.5 Recovery Flow

The complete fault-to-recovery lifecycle is illustrated below:

```
[ NORMAL STATE ]  ──>  (Voltage input drops to 0V / wire disconnected)
       |
       v
[ CRITICAL STATE ]  ──>  (Fault Manager stops petting the Watchdog)
       |
       v
[ LAST GASP WRITE ]  ──>  (MODE_CRITICAL written to EEPROM addr 0x00)
       |
       v
[ WATCHDOG BITE ]  ──>  (Hardware forces reboot after 2.0 s starvation)
       |
       v
[ .init3 BOOT HOOK ]  ──>  (MCUSR captured before crt0 wipes registers)
       |
       v
[ RECOVERY TELEMETRY ]  ──>  (System prints diagnostic banner over USB)
```

Upon reboot, Logger_Check_Previous_Death() evaluates the captured MCUSR flags and reads the EEPROM death certificate. It broadcasts a SYSTEM RECOVERY ALERT banner over USB, identifying the Watchdog reset and the prior critical state. The EEPROM address is then cleared, and the system resumes normal operation.

5. Software Module Breakdown

5.1 Source File Overview

The firmware is organized into a multi-file compilation unit with strict separation of hardware and application concerns. All source files are managed under the PlatformIO build system with the AVR-GCC toolchain.

Source File	Type	Role
include/system_types.h	Header	Global memory blueprint — TelemetryDAQ_t struct, SystemMode_t enum, volatile declarations
include/hardware_hal.h	Header	Hardware Abstraction Layer interface — function prototypes for all subsystems
src/timer_hal.cpp	Driver	Timer1 CTC configuration and TIMER1_COMPA_vect ISR — 1 ms heartbeat
src/adc_service.cpp	Service	Non-blocking ADC state machine + 8-sample circular buffer moving-average DSP
src/fault_service.cpp	Service	Watchdog supervisor — fault evaluation, WDT petting, and critical-mode detection
src/logger_service.cpp	Service	.init3 naked boot hook, MCUSR capture, EEPROM crash log read/write
src/uart_service.cpp	Driver	Native USB CDC telemetry formatting and transmission at 115200 baud
src/main.cpp	Orchestrator	Cooperative task scheduler, ATOMIC_BLOCK tick capture, CPU sleep management

5.2 Global Memory Blueprint (system_types.h)

All inter-module shared state is packed into a single TelemetryDAQ_t struct. This enforces strict data formatting and provides a single source of truth for the system's operational state. Variables modified by hardware interrupts are marked volatile to prevent the compiler from optimizing away their reads.

```
typedef enum { MODE_INIT, MODE_NORMAL, MODE_WARNING, MODE_CRITICAL } SystemMode_t;

typedef struct {
    volatile uint32_t system_tick;  // Modified by Timer1 ISR
    uint16_t temperature;           // Modified by ADC_vect ISR
    uint16_t voltage;               // Modified by ADC_vect ISR
    uint8_t  fault_code;
    SystemMode_t current_mode;
} TelemetryDAQ_t;
```

5.3 Fault State Transitions

The Fault Manager evaluates sensor readings every 100 ms and transitions between three operational states:

- MODE_NORMAL: Temperature <= 512 AND Voltage >= 100. WDT is petted normally.
- MODE_WARNING: Temperature > 512 (overheating). WDT still petted; LED goes solid ON.
- MODE_CRITICAL: Voltage < 100 (power rail failure / open circuit). WDT starvation begins; EEPROM write occurs; LED goes OFF.

5.4 Telemetry Packet Format

The system broadcasts a structured telemetry packet over USB CDC at 115200 baud every 1000 ms, in the following format:

```
[12345ms] TEMP_RAW: 487 | VOLT_RAW: 623 | SYS_MODE: NORMAL
[13345ms] TEMP_RAW: 521 | VOLT_RAW: 619 | SYS_MODE: WARNING
[14345ms] TEMP_RAW: 534 | VOLT_RAW: 12  | SYS_MODE: CRITICAL
```

The timestamp field (system_tick in ms) enables precise upstream correlation of events during post-analysis.

6. Experimental Verification and Fault Analytics

6.1 Test Methodology

Following firmware development, the DAQ node was subjected to three structured test cases designed to validate the three core reliability claims: scheduling determinism, DSP effectiveness, and autonomous fault recovery.

Test	Case Name	Methodology	Results & Observations
A	1 ms Scheduler Accuracy	Monitor LED toggle interval on Timer1 1000 ms task	LED toggled precisely on 1000 ms boundary; zero measurable drift. ATOMIC_BLOCK successfully prevented 32-bit tick tearing.
B	DSP Noise Mitigation	Inject high-frequency electrical noise on A0; compare raw vs filtered output	Signal fluctuations reduced by 18 dB. 8-sample moving average stabilized output; no false fault triggers observed.
C	Watchdog Starvation & EEPROM Recovery	Tie Voltage input (A1) directly to GND to simulate catastrophic power loss	System entered CRITICAL within 100 ms → WDT bite at 2.0 s → reboot → .init3 captured WDRF flag → EEPROM death certificate read and cleared → system resumed NORMAL operation.

6.2 Detailed Test Observations

Test Case A — Scheduler Timing Precision

The LED heartbeat task (1000 ms slot) was monitored using an oscilloscope on Port C, Pin 7. Transitions were recorded over a 60-second observation window. The maximum observed deviation from the 1000 ms target was less than 2 µs i.e a 99.98% improvement over the ±12 ms jitter observed with Arduino's millis()-based timing. This confirms that the ATOMIC_BLOCK wrapping eliminates all risk of tick tearing on the 8-bit architecture.

Test Case B — DSP Noise Reduction

High-frequency electrical noise was injected onto the A0 temperature input line. The raw ADC output showed peak-to-peak fluctuations exceeding 60 ADC counts. After the 8-sample moving-average filter, peak-to-peak variation was reduced to under 4 counts, a measured attenuation of 18 dB. This level of stability ensures that the 512-count WARNING threshold cannot be tripped by noise.

Test Case C — Watchdog Starvation and EEPROM Recovery

The voltage sensor line (A1) was hard-tied to GND, simulating an instantaneous power rail failure. The chronological event log was:

```
T+0ms    : A1 reads 0. Fault Manager triggers MODE_CRITICAL.
T+0ms    : Logger_Record_Death() writes 0x03 (MODE_CRITICAL) to EEPROM[0x00].
T+0ms    : Watchdog petting stops. WDT 2.0s countdown begins.
T+1000ms : Telemetry: SYS_MODE: CRITICAL
T+2000ms : WATCHDOG BITE — CPU resets. USB disconnects.
T+2050ms : [BOOT] .init3 captures MCUSR = 0x08 (WDRF bit set)
T+2050ms : [BOOT] EEPROM[0x00] = MODE_CRITICAL
T+2050ms : >> RECOVERY ALERT: Reset Cause: WATCHDOG BITE
T+2050ms : >> RECOVERY: Previous state was CRITICAL (Voltage Failure)
T+2050ms : EEPROM[0x00] cleared. System resuming NORMAL operation.
```

7. System Performance Metrics

The table below provides a quantitative comparison of the bare-metal implementation against the standard Arduino framework on the same ATmega32U4 hardware:

Parameter	Standard Arduino	Bare-Metal Implementation	Improvement
ADC Lockup Time	104 µs (blocking)	0 µs (background ISR)	**100% CPU overhead reduction**
Timer Jitter	±12 ms	< 2 µs	**99.98% timing accuracy**
Idle Power Draw	15.2 mA	8.4 mA (CPU Idle Sleep)	**44.7% energy savings**
Exception Recovery	Infinite hang	< 2.0 s (Watchdog reboot)	**Full autonomous recovery**

All figures were measured on the same ATmega32U4 hardware at 16 MHz. Power measurements taken with a precision ammeter at 5V supply. Timer jitter measured with an oscilloscope at 1 GSa/s sampling rate. DSP attenuation calculated from peak-to-peak ADC count variation before and after filtering.

8. Industrial Applications and Relevance

8.1 Automotive ECU Alignment

The architecture of this DAQ node closely mirrors the design philosophy of automotive Electronic Control Units (ECUs). In automotive systems, timing determinism and fault tolerance are safety-critical requirements. The cooperative scheduler, WDT-based recovery, and EEPROM event logging are standard patterns in AUTOSAR-compliant ECU firmware. This project demonstrates these patterns on a cost-effective 8-bit platform.

8.2 Industrial Sensor Networks

Industrial IoT sensor nodes share identical requirements: low power consumption, deterministic sampling intervals, noise-immune signal conditioning, and autonomous recovery from communication or power failures. The 44.7% power reduction achieved through sleep mode management directly extends battery life in remote monitoring deployments.

8.3 Motorsport and Formula SAE Telemetry

The Team Blitz Racing application context is directly relevant. In a go-kart or formula-style vehicle, the DAQ node's architecture maps cleanly onto: throttle position, temperature, and voltage acquisition (ADC state machine); real-time data logging (telemetry pipeline); and automatic recovery from electrical noise or connector vibration events (watchdog + EEPROM recovery). The non-blocking design ensures that sensor acquisition never delays the control loop.

9. Challenges and Engineering Decisions

9.1 The 32-Bit Tick Tearing Problem

Initial testing revealed subtle, intermittent timing anomalies in the scheduler. Diagnosis identified the root cause as interrupt tearing: the 1 ms Timer1 ISR firing between byte reads of the 32-bit system_tick variable, corrupting the captured timestamp. The solution ATOMIC_BLOCK(ATOMIC_RESTORESTATE) was selected over cli()/sei() pairs because it correctly handles nested interrupt contexts and guarantees state restoration even if the block is entered from within an ISR.

9.2 The .init3 Section and crt0 Bypass

Intercepting the MCUSR register required solving a classic embedded systems bootstrap problem: standard main()-entry diagnostics are too late, because crt0 has already erased the hardware flags. The .init3 linker section was identified as the correct injection point. It executes after the hardware is initialized but before any C variable initialization. The naked attribute prevents the compiler from generating function prologue/epilogue code that would corrupt the stack state during this early initialization phase.

9.3 EEPROM Write Timing vs WDT Expiry

A critical timing constraint existed between the EEPROM write operation and the WDT expiry. EEPROM write cycles on AVR silicon complete in approximately 3.4 ms. With the WDT set to 2.0 seconds, the last-gasp write had ample time to complete before the hardware reboot. A static boolean flag (death_logged) was added to ensure the EEPROM write only occurs once per critical event, preventing EEPROM wear from repeated writes during the starvation window.

10. Conclusion

This project successfully demonstrates that robust, real-time telemetry systems can be engineered on simple 8-bit microcontrollers through disciplined, layered firmware design and direct register programming.

By completely bypassing Arduino's high-level, blocking abstraction layer, the system achieved industrial-grade performance characteristics: sub-2-microsecond scheduling jitter, zero CPU overhead during analog acquisition, 18 dB of DSP noise attenuation, 44.7% idle power reduction, and fully autonomous crash detection, logging, and recovery, all on a single microcontroller.

The architecture featuring a cooperative scheduler, interrupt-driven state machines, non-volatile crash persistence, naked boot diagnostics, and hardware watchdog supervision is directly aligned with design patterns used in AUTOSAR-compliant automotive ECUs, industrial sensor network nodes, and motorsport data acquisition systems.

This project confirms a fundamental principle of embedded systems engineering: constraints are not obstacles. The memory, speed, and register limitations of 8-bit AVR silicon did not prevent the implementation of a professional-grade, fault-tolerant system, they forced the architectural clarity that made it possible.

11. References

References

[1] Microchip Technology Inc., "ATmega32U4 8-bit Microcontroller with 32K Bytes of In-System Programmable Flash," Datasheet, Document Number: 7766J-AVR-11/15, Revised November 2015.

[2] H. Kopetz, Real-Time Systems: Design Principles for Distributed Embedded Applications, 2nd ed., New York, NY: Springer, 2011.

[3] M. J. Pont, Patterns for Time-Triggered Embedded Systems: Building Reliable Applications with the 8051, Addison-Wesley, 2001.

[4] M. J. Pont, The Embedded C Template Library: Designing Reliable Embedded Systems with Time-Triggered Architectures, Automotive Edition, Leicester, UK: SafeTTy Systems, 2014.

[5] Microchip Technology Inc., "AVR Libc Reference Manual: User Manual for <util/atomic.h>," Rev. 2.0, Toolchain Documentation, 2021.

[6] Microchip Technology Inc., "An Introduction to Watchdog Timers and How to Use Them," Application Note AN2515, Chandler, AZ: Microchip Technology Inc., 2019.

[7] J. J. Labrosse, MicroC/OS-II: The Real-Time Kernel, 2nd ed., CMP Books, 2002.

[8] S. W. Smith, The Scientist and Engineer's Guide to Digital Signal Processing, San Diego, CA: California Technical Publishing, 1997, Chapter 15: "Moving Average Filters."

[9] Free Software Foundation, "Using the GNU Compiler Collection (GCC): Variable and Function Attributes," GCC Version 11.2 Manual, 2021.

[10] PlatformIO Labs, "PlatformIO Documentation — AVR Platform," PlatformIO Ecosystem Manual, 2024. [Online]. Available: https://docs.platformio.org

12. Declaration and Sign-Off

I hereby declare that this project report is a true and accurate account of the work performed during my internship. The design, implementation, and experimental verification described in this report were carried out by me under the supervision of the stated mentor.

Developer	Supervising Mentor
Name: Avyukth M A Degree: B.Tech ECE (2024–2028) Institution: VIT Chennai	Name: Mr. Somu Sundar Title: Senior Manager Organization: Yuken India Ltd